

Security Audit Skill — User Manual

A portable Claude Code skill that audits a source tree for security defects and produces a self-contained HTML report with scored findings, non-breaking remediation guidance, and the token and dollar cost of the analysis.

1. Install

The skill is a folder. Copy it to one of two places:

```
# Available in one project only
cp -r security-audit <your-project>/.claude/skills/

# Available in every project on this machine
cp -r security-audit ~/.claude/skills/
```

Windows PowerShell:

```
Copy-Item -Recurse security-audit "$HOME\.claude\skills\"
```

Then confirm Claude Code can see it:

```
/help          # the skill list includes security-audit
```

Requirements

Requirement	Notes
Claude Code	Provides the <code>Grep</code> / <code>Glob</code> / <code>Read</code> tools the scan runs on. Nothing else needed.
A shell	<code>bash</code> or <code>PowerShell</code> — you already have one. Used to assemble the report.
Cost accounting	Uses whichever of <code>Python</code> / <code>bash+awk</code> / <code>PowerShell 5.1</code> is present. All three give identical numbers. Without any of them the audit still runs and the report shows cost as "not collected".
Network / MCP	Not required. Only improves dependency-advisory checking — see §7.

Nothing to install. No `pip install`, no `npm install`, no build step, no interpreter to fetch. The skill will never interrupt an audit to ask you to install something.

```
Python is optional. If you happen to have it, the skill uses it for cost accounting; if you don't, it uses
awk or PowerShell instead and the output is identical. Python is only genuinely required to develop the
skill — running the rule-pack tests — never to use it. See §10.
```

2. Run it

```
/security-audit ./src
```

That is the whole minimum. The skill will inventory the tree, detect the stack, run the applicable rule packs, triage the results, score the app, write a remediation plan, measure what the run cost, and drop the report next to your code.

Real examples

```
# Audit a generated Blazor app, name it explicitly
/security-audit ./Test/Demo-3/Informix-demo-3 --name "Informix-demo-3"

# Quick pre-commit sanity pass, findings only, no remediation plan
/security-audit ./api --depth quick --mode analyze

# Full pre-production review, report to a specific folder
/security-audit /srv/app --depth deep --out ~/reports

# Re-audit after fixing things, and diff against the previous run
/security-audit ./src --baseline ./security-reports/.security-audit/findings.json

# Skip the code the audit shouldn't judge
/security-audit . --exclude "**/legacy-vendor/**,**/*.generated.cs"
```

You can also just ask for it in words — *"run a security audit on ./src"* — and the skill triggers.

3. Options

Option	Default	What it does
<code><target-path></code>	required	Directory or file to audit. Everything outside it is off limits.
<code>--name "<Project>"</code>	inferred from the folder	Used in the report title and filename.
<code>--out <dir></code>	<code><target>/../security-reports</code>	Where the report is written.
<code>--depth</code> <code>quick standard deep</code>	<code>standard</code>	How much analysis effort. See below.
<code>--mode</code> <code>analyze plan</code>	<code>plan</code>	<code>analyze</code> = findings only. <code>plan</code> adds the remediation roadmap and the projected score.
<code>--exclude "<globs>"</code>	—	Extra comma-separated globs to skip, on top of the built-in exclusions.
<code>--baseline</code> <code><findings.json></code>	—	A previous run to compare against; findings come back tagged New / Persisting / Resolved.
<code>--no-cost</code>	off	Skip cost measurement.

If you omit the path, the skill asks for it. It will not default to the current directory — auditing the wrong tree costs real money.

Choosing a depth

Depth	What it does	Relative cost	Use it for
<code>quick</code>	Universal + detected-stack rules; evidence gathered for Critical and High only	1×	CI gates, a fast check after fixes
<code>standard</code>	All matching rule packs; every hit triaged; auth and data-flow read of the entry points	~3×	Default. Release reviews.
<code>deep</code>	The above, plus every entry point, trust boundary, auth path and data sink read by hand; business-logic review	~8×	Pre-production, regulated workloads, the first audit of a codebase

The multipliers are relative to each other, not absolute — actual cost scales with codebase size. Run `standard` first; the report's cost section then tells you what `deep` would roughly cost on *your* code.

4. What you get

One file, named exactly:

```
<Project name> - Security analysis report. - <YYYY-MM-DD>.html
```

for example `Informix-demo-3 - Security analysis report. - 2026-09-09.html`.

Open it in any browser by double-clicking. It is fully self-contained — no internet, no server, no assets folder. You can email it, attach it to a ticket, or drop it in SharePoint and it still works.

Alongside it, `<out>/security-audit/` holds the working files: `findings.json` (the structured data, useful for CI and for `--baseline` on the next run), `cost.json`, `inventory.json` and `raw-hits.json`.

Reading the report

Section	What it is for
Score dashboard	Current score, projected score after remediation, and the uplift. Severity counts.
Category breakdown	Per-OWASP-category scores and weights, as bars and as a table. Categories that were not assessed are marked as such — they are not scored as passing.
Controls verified present	What is already correct. Read this before remediating so you don't undo it.
Findings summary	Every finding with severity, confidence, OWASP category and location, plus a working Fixed checklist.
Detailed findings	One card per finding: standards mapping, risk, evidence with <code>file:line</code> , impact, remediation steps, a before/after diff, and a verification step.
Remediation roadmap	Phased plan ordered by risk reduction per unit of effort, with the projected score after each phase.
Analysis cost	Tokens and dollars for this run, by model.

Section	What it is for
Scan evidence	What was checked — so "no finding" can be told apart from "not checked".
Limitations & scope	What the audit could <i>not</i> see. Read this section.

Things worth knowing

- **Filter and search.** Severity chips and a search box above the findings. Search covers titles, filenames, CWE ids and evidence text.
- **The Fixed checklist persists** in that browser, so you can work through it across sessions. It is a working aid, not a record — re-run the audit to get a verified post-remediation score.
- **Copy checklist** puts the whole findings list on your clipboard as Markdown, ready to paste into a ticket or a PR description.
- **Print to PDF** works properly — Ctrl/Cmd-P expands every finding and hides the controls. Use it when you need the `.pdf` the original design plan called for.
- **Theme.** Follows your OS setting; the button in the corner overrides it.

5. Understanding severity and confidence

Severity is impact in *your* application, not the generic reputation of the pattern:

Severity	Means
Critical	Unauthenticated remote compromise, mass data exposure, or full auth bypass. Exploitable now, by anyone who can reach the app.
High	Significant compromise, but needs a precondition — an unprivileged account, a specific input, user interaction.
Medium	A control meaningfully weakened, or a defense-in-depth failure that needs another flaw to matter.
Low	Hardening gap, limited direct impact.
Info	Observation or policy gap, not directly exploitable.

Confidence tells you how much verification stands behind it:

Confidence	Means	What to do
Confirmed	The code was read; the flaw is present and reachable	Treat as real
Likely	Strong evidence, one assumption unverifiable from the code	Check the stated assumption
Possible	Needs human knowledge the audit didn't have	Answer the question stated in the finding

Confidence also scales the score, so a `Possible` finding moves the number less than a `Confirmed` one. If you resolve a `Possible` either way, re-run to get an accurate score.

Posture can be worse than the score suggests

A single Critical finding forces the posture band to **At Risk** even at 91/100. That is deliberate: one unauthenticated admin endpoint is not a hardened application, whatever the weighted average says.

6. Cost

Every report states what the analysis cost, measured from the session's actual usage records — tokens by type (input, cache write, cache read, output) and dollars per model.

Read the `source` label:

Label	Meaning
<i>(no caveat shown)</i>	Measured from real usage records
"Estimated, not measured"	Usage records were unavailable; treat as approximate
"not collected"	Cost was not measured at all

Two things to keep in mind:

- **Dollar figures are list rates.** They exclude enterprise discounts, Batch API pricing, and Bedrock/Vertex partner rates. Edit `security-audit/assets/pricing.json` to match your actual agreement — one line per model.
- **Total tokens and total cost don't move together.** Cache reads bill at a small fraction of the input rate, so a run with millions of cached tokens can cost less than a run with far fewer fresh ones.

To keep costs down: `--depth quick` for routine checks, point the path at the subtree you actually changed rather than the repo root, and use `--mode analyze` when you only need the findings.

7. Dependency checking, MCP, and network access

The audit needs no MCP server and no network access. Scanning, triage, scoring, reporting and cost accounting are all local.

One thing genuinely benefits from network access: confirming whether your declared dependencies have known CVEs. The skill handles this in order of preference:

1. **An advisory MCP server**, if your organization has one (GitHub Advisory, OSV, Snyk, Sonatype). Enable it in your Claude Code MCP configuration before running; the skill will use it and say so.
2. **Native tooling**, if present and you approve it — `dotnet list package --vulnerable`, `npm audit`, `pip-audit`, `govulncheck`. **The skill asks first**, because these run your project's tooling and may reach the network.
3. **WebFetch against the public OSV API** (`https://api.osv.dev/v1/query`), if you allow it.
4. **Nothing.** The report then inventories your dependencies and files a finding for `"no automated dependency scanning"` — and the Limitations section states plainly that advisories were **not** verified.

That last point matters: if the audit could not check your dependencies, the report says their status is **unknown**, not clean. Don't read a quiet dependency section as a pass — check the Limitations section.

No MCP server ships with the skill, and none is enabled automatically.

8. Running it in CI

Headless, with an exit-code gate:

```
claude -p "/security-audit ./src --depth quick --mode analyze --out ./security-reports"

# fail the build on any Critical finding
grep -q '"severity": "Critical"' ./security-reports/.security-audit/findings.json \
  && { echo "Critical security finding - failing build"; exit 1; }

exit 0
```

Notes for CI use:

- `--depth quick` keeps the cost predictable on every push. Schedule a `standard` or `deep` run weekly or per release instead of per commit.
- Publish the HTML file as a build artifact — it is self-contained, so artifact viewers render it.
- Keep the previous run's `findings.json` and pass it as `--baseline` so the job can report *new* findings rather than the whole backlog.
- Cost measurement needs the session transcript, which exists in `claude -p` runs too.

9. Troubleshooting

Symptom	Cause	Fix
The report opens blank	<code>findings.json</code> was not valid JSON	The page shows the parse error at the top. Validate <code>.security-audit/findings.json</code> and re-concatenate the three files.
Cost section says "not collected"	No session transcript was found, or none of the three collectors could run	Usually the transcript. Pass <code>--transcript <file></code> explicitly to whichever collector you use (<code>cost.py</code> , <code>collect-cost.sh</code> , <code>collect-cost.ps1</code>), or accept it — the audit itself is unaffected.
Cost figure looks ~30× too high	A collector fell back to scanning the whole transcript instead of just this run's lines	Check <code>.security-audit/cost-watermark</code> exists and its path is readable by the collector you ran. Mixing shells can cause this: Git Bash writes <code>/c/Users/...</code> , which only <code>cost.py</code> translates. Stick to one collector per run.
Cost looks far too high	You are reading total <i>tokens</i> , not cost	Cache reads dominate token counts but bill at a fraction of the rate. Check the dollar column.
A category shows "Not assessed"	That category genuinely was not evaluated	Read its note. Usually a missing rule pack for the language, or unreachable tooling. This is the honest result, not a bug.
A finding looks like a false positive	Triage got it wrong, or context the skill couldn't see	Check the <code>confidence</code> field first. <code>Possible</code> findings are flagged as needing your confirmation. Tell the skill what it missed and re-run; consider adding the exclusion to <code>--exclude</code> .
Report has no findings at all	Small or non-application code, or an unsupported language	Check Limitations and the Scan evidence table — they show whether anything was actually checked.

Symptom	Cause	Fix
Score seems unfairly low	One Critical dominates the weighted category	Read the roadmap: Phase 1 usually recovers most of the score.
My language isn't covered	No rule pack matches	The universal pack still applies. Add a pack — see below.

10. Python: what it is and isn't used for

The skill runs with nothing installed. Python, if present, is used as an accelerator — and it is required only for maintenance work.

Task	Needs Python?
Running an audit	No
Producing the HTML report	No
Cost accounting	No — Python is preferred, awk and PowerShell are equal fallbacks
Editing rule packs and running the tests	Yes — <code>tests/run_corpus.py</code>

How the skill finds Python

`bin/ensure_python.sh` (or `.ps1`) checks, in order: `$SECURITY_AUDIT_PYTHON` → `PATH` → well-known install locations and the registry `PATH` → a previously cached copy → an opt-in download.

It **probes by executing code** rather than trusting `PATH`, for two reasons that bite constantly on Windows:

- `%LOCALAPPDATA%\Microsoft\WindowsApps\python.exe` is a **0-byte App Execution Alias**. It resolves on `PATH`, prints a Microsoft Store advert, and exits non-zero. Anything that trusts `where python` picks it and breaks.
- A correct `PATH` in the registry is **invisible to an already-running process** that started before Python was installed. Restarting your editor fixes it; until then, the well-known-locations search finds the interpreter anyway.

To point it at a specific interpreter:

```
export SECURITY_AUDIT_PYTHON=/usr/local/bin/python3.12      # macOS / Linux
$env:SECURITY_AUDIT_PYTHON = "C:\Python312\python.exe"      # Windows
```

The opt-in download

If no Python exists and you *want* one, `ensure_python.sh --allow-download` fetches a pinned CPython build (~30 MB) into `~/.claude/cache/security-audit/python/`. Three properties make this safe:

- **Pinned and hash-verified.** The exact version and its SHA-256 live in `assets/python-bootstrap.json`. The archive is checked *before* extraction; a mismatch fails closed and deletes the download.
- **Outside your repository.** It lands in the user cache — never in the skill folder, never in your project.
- **Never automatic.** The skill asks first and tells you what it will download. Declining costs you nothing.

This is deliberately stricter than a typical installer script. The skill's own `CFG-CI-02` rule flags curl-piped-to-shell and unpinned dependencies as findings — a security tool that did the thing it warns about would not deserve to be believed.

Running the rule-pack tests

```
python tests/run_corpus.py
```

Checks that every pattern compiles under ripgrep's Rust regex engine, and that rules fire on the vulnerable fixtures **and stay silent on the safe ones**. Run it after any change to `rules/*.json`. The silence half is the one that matters — a rule that flags parameterized SQL trains people to ignore the whole report.

11. Extending the skill

Adding rules for your stack

Rule packs are plain JSON in `security-audit/rules/`. Copy the closest one, edit the patterns, and register it in `00-index.json` with an `appliesTo` and `detect` list.

Two constraints, both from ripgrep's Rust regex engine:

- **No lookahead/lookbehind** (`(?!...)`, `(?<=...)`) and **no backreferences** (`\1`).
- Escape literal braces: `interface\{\}`.

Validate before relying on a new pattern:

```
printf 'x\n' | rg -e '<your pattern>' # exit 2 means the regex does not compile
```

Give every rule a `falsePositiveHints` field describing what would *close* the finding. That field is what keeps the report precise, and it is the difference between a report someone acts on and a wall of noise they learn to ignore.

12. Good practice

- **Audit the smallest meaningful scope.** Point at `./src`, not the repo root with its `node_modules` and build output.
- **Read Limitations first.** It tells you what the score does *not* cover.
- **Fix in roadmap order.** Phase 1 is chosen for risk reduction per unit of effort, and access control fixes often close several findings at once.
- **Run each remediation's verification step.** They are written to be runnable, and a fix nobody verified is a fix nobody has.
- **Re-run after fixing.** The projected score is a projection. Only a fresh run gives you a measured post-remediation score.
- **Keep `findings.json`.** It is your baseline for the next run and your input for tracking.

- **Treat committed secrets as already compromised.** If the report finds a credential in source, rotate it. Deleting it from `HEAD` does not un-leak it — it stays in git history and in every clone and CI log.